

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Liczniki

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 6

Studia dzienne

Informatyka

Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 07.04.2022

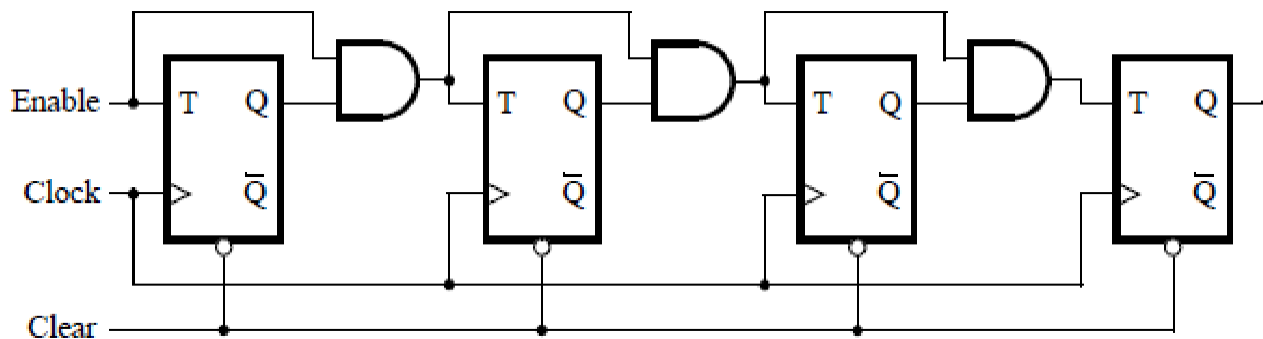
Zadanie 1

Utwórz projekt counter_T_4_bits dla 4-bitowego licznika na przerzutnikach T z następującym przyporządkowaniem pinów:

Wyprowadzenia	Piny
clk	KEY[0]
areset	SW[0]
enable	SW[1]
Q	HEX0

Gdzie: **clk** – sygnał zegara;
areset – asynchroniczne ustawianie;
enable – zezwolenie na zliczanie;
Q – rezultat.

Schemat układu:



Opis działania przerzutnika T:

```
always @(posedge clk, negedge aclr)
    if (!aclr)    Q <= 1'b0;
    else if (T)   Q <= ~Q;
    else         Q <= Q;
```

Kod w Verilog

Moduł 4-bitowego licznika na przerzutnikach T

```
module counter_T_4_bits(
    input clk, areset, enable,
    output [3:0] Q);

    wire [2:0] c;
    assign c[0] = Q[0] & enable;
    assign c[1] = Q[1] & c[0];
    assign c[2] = Q[2] & c[1];
    FFT_areset fft0(enable, clk, areset, Q[0]);
    FFT_areset fft1(c[0], clk, areset, Q[1]);
    FFT_areset fft2(c[1], clk, areset, Q[2]);
    FFT_areset fft3(c[2], clk, areset, Q[3]);

endmodule
```

Moduł przerzutnika T

```

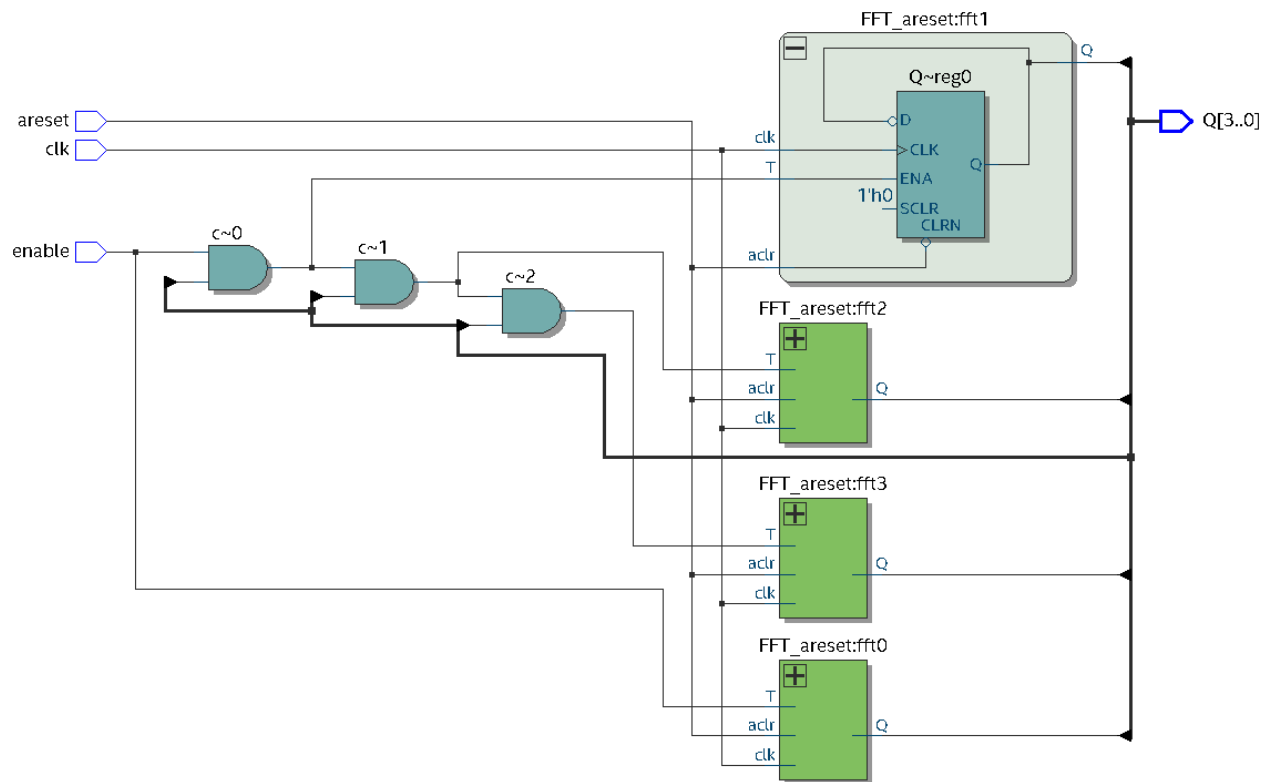
module FFT_areset(
    input T, clk, aclr,
    output reg Q);

    always @ (posedge clk, negedge aclr)
        if (!aclr) Q <= 1'b0;
        else if (T) Q <= ~Q;
        else Q <= Q;

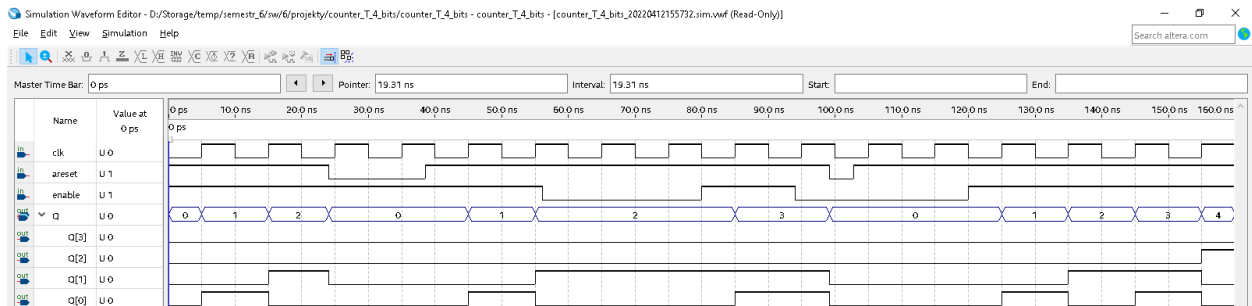
endmodule

```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Zadanie 2

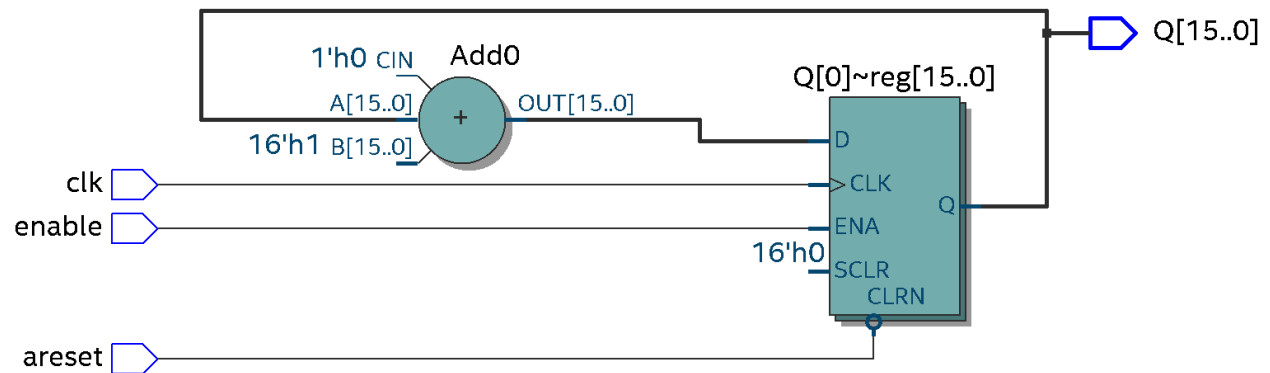
Utwórz projekt counter_16_bits 16-bitowego licznika, wykorzystując jego behawioralny/algoritmiczny opis za pomocą operatora: $Q \leq Q + 1$. Wyświetl wynik na wyświetlaczach HEX3-0.

Kod w Verilog

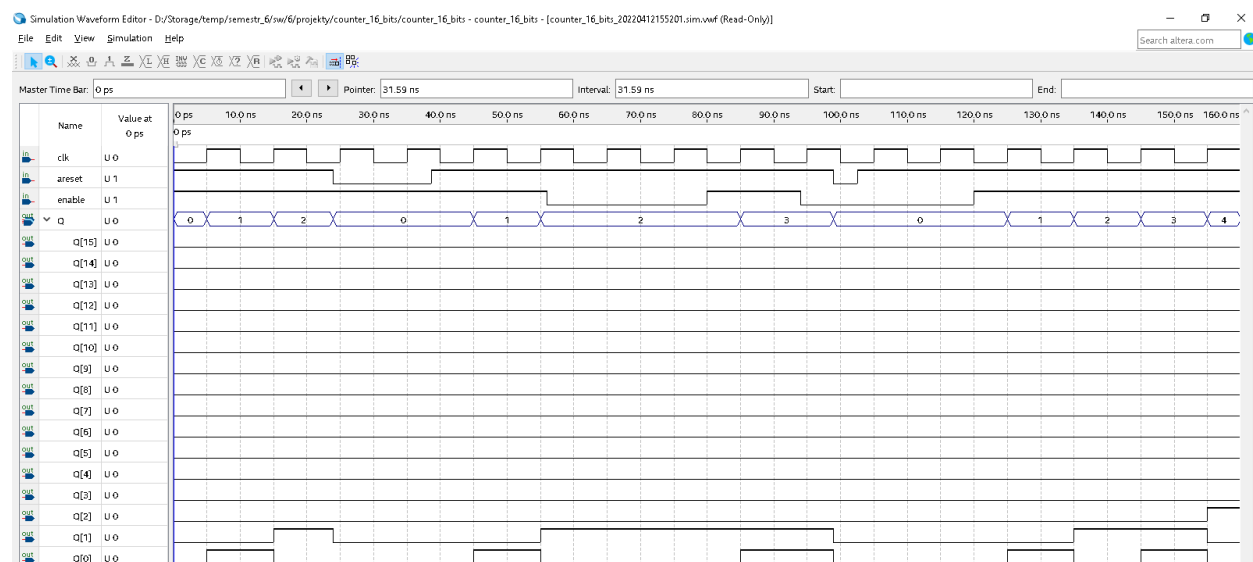
Moduł 16-bitowego licznika

```
module counter_16_bits(  
    input clk, areset, enable,  
    output reg [15:0] Q);  
  
    always @ (posedge clk, negedge areset)  
        if (!areset) Q <= {16{1'b0}};  
        else if (enable) Q <= Q + 1'b1;  
        else Q <= Q;  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



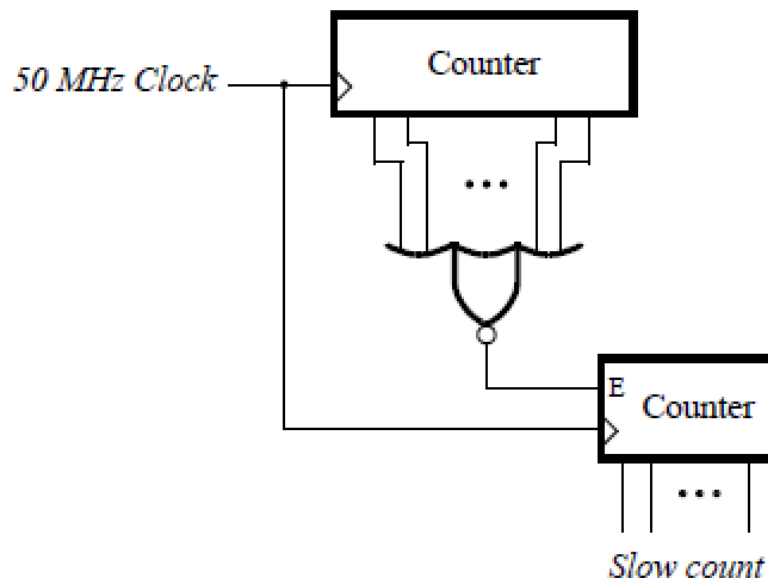
Zadanie 3

Utwórz moduł licznika modulo `counter_mod_M`, który kolejno wyświetla liczby od 0 do 9 na wyświetlaczu HEX0 z interwałem 1 s.

Wskazówka: skorzystaj z diagramu na Rys. 1, aby zmniejszyć częstotliwość synchronizacji licznika. Aby to zrobić, utwórz projekt licznika `counter_N_bits` na bitach N, a następnie opracuj projekt licznika według modułu `counter_mod_M`, który zlicza do wartości M. (Przy ustalaniu liczby bitów licznika według modułu, użyj funkcji języka Verilog).

```
localparam N=clogb2(M-1);  
function integer clogb2(input [31:0] v);  
    for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)  
        v = v >> 1;  
endfunction
```

Ustal wartość liczby M, aby licznik liczb od 0 do 9 działał w odstępie 1 sekundy.



Rys. 1. Tworzenie wolnego licznika

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module slow_counter_mod_10_on_board(  
    input CLOCK_50,  
    output [6:0] HEX0);  
  
    wire [3:0] Q;  
    slow_counter_mod_10 slow_counter(CLOCK_50, Q);  
    decoder_hex_10 decoder(Q, HEX0);  
endmodule
```

Moduł dekodera z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_10(
    input [3:0] binary,
    output reg [6:0] h);

    always @ (*)
        case (binary)
            4'd0: h = 7'b1000000; // 64
            4'd1: h = 7'b1111001; // 121
            4'd2: h = 7'b0100100; // 36
            4'd3: h = 7'b0110000; // 48
            4'd4: h = 7'b0011001; // 25
            4'd5: h = 7'b0010010; // 18
            4'd6: h = 7'b0000010; // 2
            4'd7: h = 7'b1111000; // 120
            4'd8: h = 7'b0000000; // 0
            4'd9: h = 7'b0010000; // 16
            default: h = 7'b1111111;
        endcase

endmodule
```

Moduł licznika modulo 10 z interwałem 1 s. Oscylator kwarcowy (zegar CLOCK_50) w płycie DE1-SoC ma częstotliwość 50 MHz, więc szybki licznik fast_counter jest sparametryzowany na modulo 50 000 000.

```
module slow_counter_mod_10(
    input clk,
    output [SLOW_COUNTER_BITS-1:0] Q);

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
        endfunction

    // ceil(log2(50_000_000)) bitów
    localparam FAST_COUNTER_BITS = clogb2(50_000_000 - 1);
    // ceil(log2(10)) bitów
    localparam SLOW_COUNTER_BITS = clogb2(10 - 1);

    wire [FAST_COUNTER_BITS-1:0] A;
    counter_mod_M #(50_000_000) fast_counter(clk, 1'b1, 1'b1, A);
    wire e = ~|A; // e = nor(A[0], A[1], ...);
    counter_mod_M #(10) slow_counter(clk, 1'b1, e, Q);

endmodule
```

Moduł licznika modulo M

```
module counter_mod_M
    #(parameter M = 16) ( // okres licznika
    input clk, aclr, enable,
    output reg [BITS-1:0] Q);

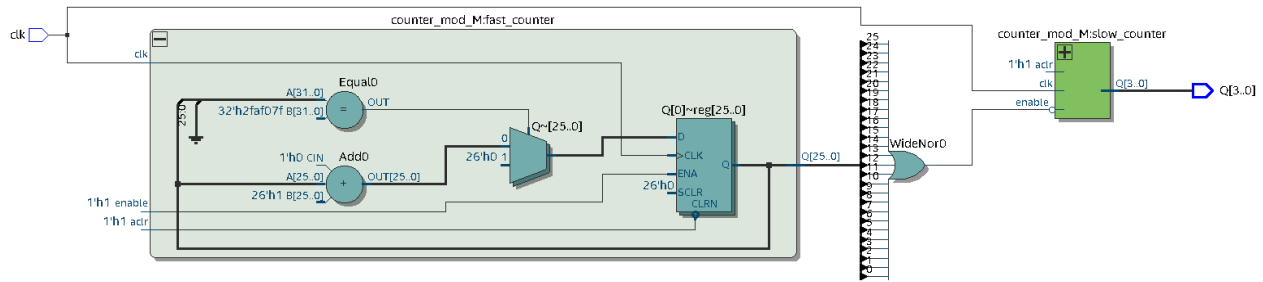
    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
        endfunction

    localparam BITS = clogb2(M-1); // długość licznika w bitach

    always @ (posedge clk, negedge aclr)
        if (!aclr) Q <= {BITS{1'b0}};
        else begin
            // zanim sprawdzimy, czy Q == M-1 i ewentualnie wyzerujemy licznik, sprawdzamy, czy na wejściu zezwalającym jest 1
            if (enable) begin
                if (Q == M-1) Q <= {BITS{1'b0}};
                else Q <= Q + 1'b1;
            end
            else Q <= Q;
        end

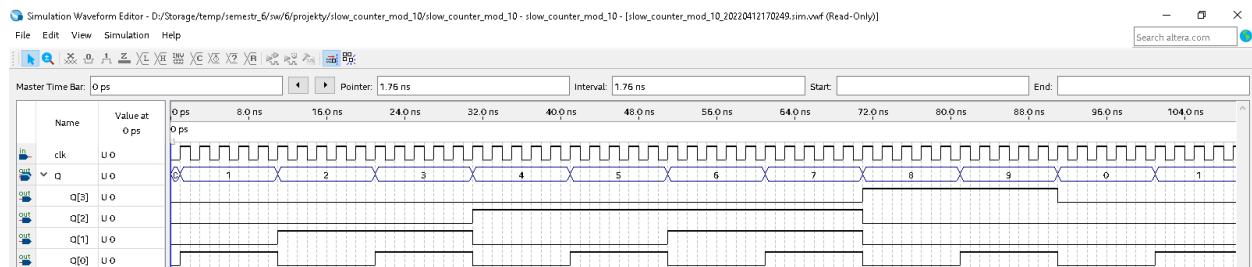
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Aby w symulacji było widać działanie wolnego licznika, parametryzujemy szybki licznik jako modulo 5. Wówczas wartość Q wolnego licznika zwiększa się o 1 co 5 zboczy narastającego zegara clk.



Zadanie 4

Wyświetl słowo ('dE1 ') na czterech 7-segmentowych wyświetlaczach. Kody wyświetlanych symboli przekształć za pomocą modułu **decoder_7_seg**. Obróć słowo na 4 wyświetlaczach w odstępach 1 sekundy w lewo lub w prawo.

Kod (c[1:0])	Symbol (h[0:6])
00	d
01	E
10	1
11	spacja

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module word_rotate_on_board(  
    input CLOCK_50,  
    output [6:0] HEX3, HEX2, HEX1, HEX0);  
  
    wire [7:0] chars;  
    word_rotate wr(CLOCK_50, chars[7:0]);  
    decoder_7_seg dec3(chars[7:6], HEX3);  
    decoder_7_seg dec2(chars[5:4], HEX2);  
    decoder_7_seg dec1(chars[3:2], HEX1);  
    decoder_7_seg dec0(chars[1:0], HEX0);  
  
endmodule
```

Moduł dekodera z kodu znaków 'o', 'd', 'E', 'S' na kod wyświetlacza 7-segmentowego

```
module decoder_7_seg(  
    input [1:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            2'b00: h = 7'b0100011; // 35; 'o'  
            2'b01: h = 7'b0100001; // 33; 'd'  
            2'b10: h = 7'b0000110; // 6; 'E'  
            2'b11: h = 7'b0010010; // 18; 'S'  
            default: h = 7'b1111111; // 127; ' '  
        endcase  
  
endmodule
```


Moduł przechowujący 4 2-bitowe kody znaków i obracający je (przesuwający cyklicznie) w odstępach 50 000 000 cykli zegara clock. Oscylator kwarcowy (zegar CLOCK_50) w płycie DE1-SoC ma częstotliwość 50 MHz, więc słowo jest obracane w odstępach 1 sekundy.

```
module word_rotate(
    input clock,
    output reg [7:0] char_codes);

    initial char_codes = { 2'b00, 2'b01, 2'b10, 2'b11 };

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

    // ceil(log2(50_000_000)) bitów
    localparam FAST_COUNTER_BITS = clogb2(50_000_000 - 1);

    wire [FAST_COUNTER_BITS-1:0] A;
    counter_mod_M #(50_000_000) fast_counter(clock, 1'b1, 1'b1, A);
    // przy zmianie A = 4 -> 0 slow_counter = 0 -> 1
    // przy zmianie A = 0 -> 1 slow_counter = 1 -> 0
    wire slow_clock = ~|A; // e = nor(A[0], A[1], ...);
    // cykliczny rejestr przesuwany o długości 4 i szerokości 2-bitów
    always @ (negedge slow_clock) begin
        char_codes[7:6] <= char_codes[1:0];
        char_codes[1:0] <= char_codes[3:2];
        char_codes[3:2] <= char_codes[5:4];
        char_codes[5:4] <= char_codes[7:6];
    end
endmodule
```

Moduł licznika modulo M

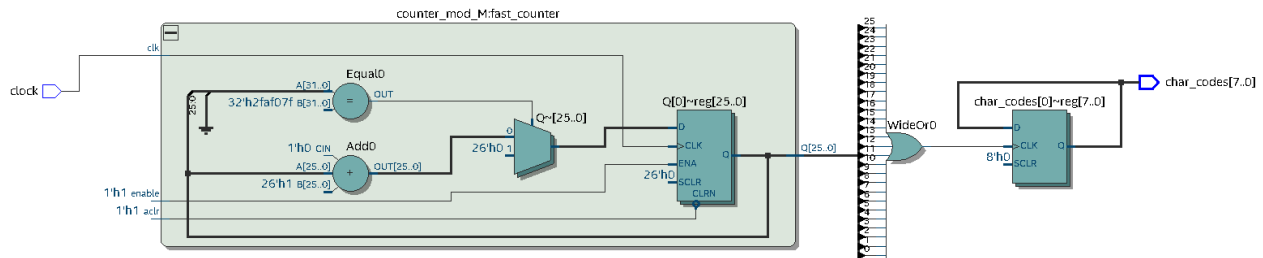
```
module counter_mod_M
#(parameter M = 16)( // okres licznika
    input clk, aclr, enable,
    output reg [BITS-1:0] Q);

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

    localparam BITS = clogb2(M-1); // długość licznika w bitach

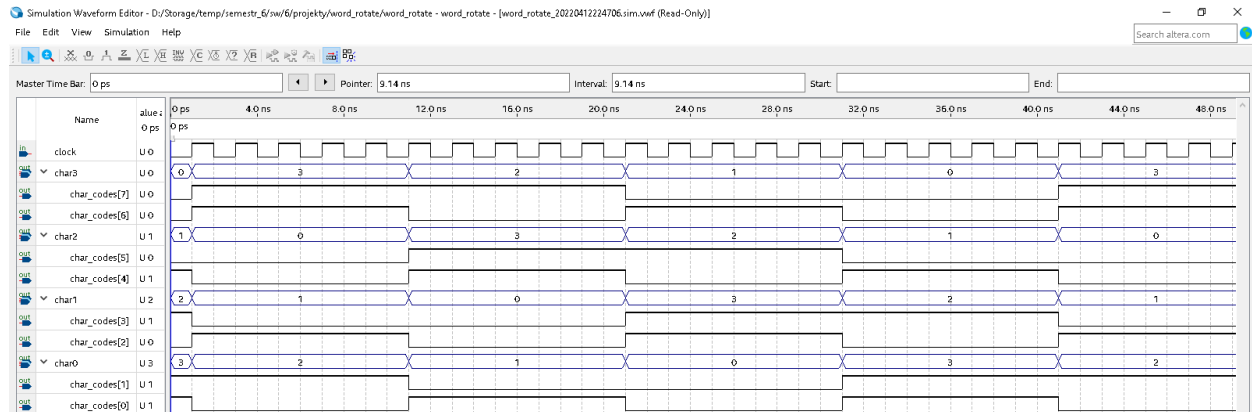
    always @ (posedge clk, negedge aclr)
        if (!aclr) Q <= {BITS{1'b0}};
        else begin
            // zanim sprawdzimy, czy Q == M-1 i ewentualnie wyzerujemy licznik, sprawdzamy, czy na wejściu zezwalającym jest 1
            if (enable) begin
                if (Q == M-1) Q <= {BITS{1'b0}};
                else Q <= Q + 1'b1;
            end
            else Q <= Q;
        end
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Aby w symulacji było widać działanie modułu obracającego słowo, parametryzujemy szybki licznik fast_counter jako modulo 5. Wówczas kody znaków przesuwają się cyklicznie co 5 zbroczy narastających zegara clock.



Wnioski

Wszystkie zadania zostały zrealizowane.